HI-WHIRLS: A Real-Time FPGA Architecture for Signal Labeling in Spectrum Sensing

Connor Steigerwald, Tianyi Zhao, Benjamin W. Domae, Johnny Zhang, and Danijela Cabric
Electrical and Computer Engineering Department, University of California, Los Angeles, USA
Email: csteigerwald1@ucla.edu, zhaotianyi@ucla.edu, bdomae@ucla.edu, jzhang566@g.ucla.edu, danijela@ee.ucla.edu

***Abstract*—Wideband spectrum sensing is essential for detecting interference, tracking spectrum occupancy, and upholding wireless security. Accurate signal detection, localization, and labeling provides the information needed to distinguish signal types, sources, and behaviors. However, software-based processing requires excessive latency and data storage. Real-time signal labeling is needed to identify and characterize signals as they occur. In this paper, we propose an efficient wideband signal detection and labeling algorithm and introduce HI-WHIRLS, a real-time FPGA-based implementation of the algorithm. The proposed algorithm is designed based on the WHIRLS algorithm, with significant improvement in processing latency and low-SNR performance. For each detected signal within a wideband spectrum, HI-WHIRLS generates precise time-frequency boundaries using configurable time and frequency resolutions. We implement HI-WHIRLS on a Xilinx RFSoC ZCU216 and evaluate it quantitatively on simulated datasets and qualitatively on real RF signals. Our experiments show that the proposed HI-WHIRLS can perform real-time RF signal detection and labeling at over a 50 MHz bandwidth with a latency of 38.58 μs and generate reliable labels at SNRs as low as -10 dB, thus providing near-instant spectrum activity insights.**

***Index Terms*—Digital signal processing, wideband spectrum sensing, FPGA, RFSoC, signal localization**

I. Introduction

Wideband spectrum sensing is a fundamental requirement for modern wireless security, electronic warfare, signal intelligence, and spectrum awareness systems. In contested or congested radio frequency (RF) environments, a sensing system must detect, localize, and characterize unknown transmissions as they occur. In these systems, operators must identify unexpected transmitters, detect interference sources, and track spectrum occupancy. However, simple energy detection is often insufficient for these applications. A more useful sensing system produces signal awareness with labels that describe a transmission's time and frequency characteristics.

Labeling signals in real-time is essential to many spectrum awareness applications that require detections while transmissions are still active. This includes rapid localization of frequency hopping emitters, or real-time channel switching and filtering. Software based algorithms can generate accurate labels, but often require storing large RF captures or transferring high-rate sample streams to a processor before labels can be produced. For real-time spectrum awareness, it is more efficient to process samples as they arrive and output signal metadata directly.

979-8-3195-1048-8/27/$31.00 © 2027 IEEE

Signal-level labeling is particularly important in wideband RF environments containing signals with different bandwidths, durations, SNRs, and spacing. Detecting that energy is present only indicates that some RF activity exists. Accurate time and frequency labels provide a detailed description of each transmission and provide input data for further analysis such as signal classification, modulation recognition, and RF fingerprinting.

In this work, we propose HI-WHIRLS, a real-time FPGA architecture for wideband signal detection and labeling based on WHIRLS [1]. HI-WHIRLS is implemented on a Xilinx RFSoC ZCU216, where it performs overlapped spectral processing, temporal averaging, detection, time-frequency grouping, and label generation in programmable logic, avoiding the need to buffer large RF captures. The main contributions of this work are as follows:

- We present HI-WHIRLS, a real-time FPGA architecture for wideband RF signal detection and labeling. The design includes support for configurable time and frequency resolutions and adaptive temporal averaging to improve low-SNR detection performance.
- We implement the complete pipeline on a Xilinx RFSoC ZCU216 evaluation board to sustain continuous RF input streams.
- We quantitatively evaluate HI-WHIRLS on a simulated dataset, achieving near unity detection probability for -12 dB SNR signals and time and frequency RMSE below 0.1 ms and 1 MHz, respectively.
- We qualitatively evaluate HI-WHIRLS on real RF captures, demonstrating detection and labeling of all transmissions in the tested examples at -10 dB SNR.

II. Background and Related Work

In this work, we study wideband signal detection and localization without prior knowledge of signals of interest. Typical classical approaches for signal detection, like matched filtering [2] or cyclostationary detection [3], require prior information about potential waveforms. Newer deep-learning methods can learn more complex signal features, but require training data and may not generalize to changing RF environments [4]. Energy detection, though, can support wideband monitoring without assuming a specific waveform or protocol

[5], [6]. Signal localization estimates each transmission's start and end times, center frequency, and bandwidth. [7], [8], isolating signal regions for radio frequency machine learning (RFML) tasks such as modulation recognition, signal classification, and RF fingerprinting [9]–[11].

Several prior algorithms perform RF signal localization. Searchlight detects and localizes signals by applying convolution kernels to detect regions of elevated energy on channograms. [12]. RISE applies image processing to a spectrogram, using adaptive thresholding and connected-component labeling to generate signal bounding boxes. [13]. Both methods operate on two-dimensional representations before producing signal labels.

HI-WHIRLS builds on the WHIRLS algorithm, which estimates time-domain and frequency-domain signal boundaries separately [1]. WHIRLS uses time-domain energy detection to identify candidate signals, estimates their frequency boundaries, and then iteratively filters the signals to refine their start and end times. Thus, WHIRLS allows independently configurable time and frequency resolutions, improving accuracy. However, WHIRLS' software implementation requires substantial data storage and processing overhead. HI-WHIRLS adapts this approach as a streaming FPGA architecture for real-time signal detection and labeling.

Real-time spectrum sensing has been previously demonstrated on SDR and FPGA platforms [14]–[17], where these systems primarily identify occupied frequency bands. Examples include adaptive channel occupancy detection under strong interference [16] and real-time estimation of signal center frequency and bandwidth [17]. However, they generally do not produce complete time-frequency labels that include signal start and end times. More recent implementations deploy machine-learning models on hardware for tasks such as CNN-based subband occupancy detection and quantized segmentation of time-frequency images [18]–[20]. These methods support more detailed spectrum analysis but depend on representative training data.

RISE and WRIST have also demonstrated real-time localization using software-based processing [9], [13]. WRIST applies a deep-learning object detector to jointly localize and classify RF signals. RISE runs an image-processing pipeline on a multicore CPU, while WRIST uses GPU-based neural-network detection. Both operate on accumulated time-frequency representations before producing labels. On the other hand, HI-WHIRLS instead processes incoming samples continuously in FPGA logic, reducing buffering and latency between signal acquisition and label generation.

III. HI-WHIRLS Algorithm

A. Algorithm Overview

HI-WHIRLS converts a continuous stream of complex IQ samples into labels describing the time and frequency boundaries of detected transmissions. The algorithm consists of noise calibration, overlapped spectral processing, adaptive thresholding, frequency-domain grouping, and temporal tracking, as shown in Fig. 1. Unlike spectrogram-based localization methods, HI-WHIRLS processes each FFT frame as it becomes available rather than waiting for a complete spectrogram to be accumulated.

B. Noise Calibration

HI-WHIRLS begins with an initial calibration interval. Input IQ samples are first divided into *frames* of size N IQ samples used for one FFT. The calibration interval consists of M_{cal} FFT frames. The average time-domain IQ value is calculated over this interval and subtracted from the input stream to reduce DC leakage. Each frame is multiplied by a Hamming window and transformed into the frequency domain. The power in each FFT bin is averaged across the calibration frames to estimate the background spectrum, denoted by $\overline{P}_{\mathrm{cal}}[k]$ for bin k. The algorithm then identifies the quietest portion of the calibrated spectrum and uses it as a global noise reference. The quietest portion is calculated from the minimum average power across a configurable group of neighboring FFT bins. For a configurable noise-reference group size G_N, the global noise reference P_N is

$$P_N = \min_k \left(\frac{1}{G_N} \sum_{i=0}^{G_N-1} \overline{P}_{\mathrm{cal}}[k+i] \right). \tag{1}$$

C. Overlapped Spectral Processing

After noise calibration, the IQ stream frames are overlapped. The N-sample frames separated by a configurable hop size H, giving an overlap fraction $\rho = 1 - \frac{H}{N}$. Each frame is multiplied by a Hamming window before the FFT to reduce spectral leakage. At sampling rate f_s, the frequency bin spacing is $\Delta f = f_s/N$, and the temporal update interval is $\Delta t = H/f_s$. Reducing the hop size produces more frequent updates without requiring a shorter FFT.

D. Adaptive State Detection

HI-WHIRLS adapts its detection parameters based on the total received power across the monitored band, using three operating states: `IDLE`, `SIGNAL_ACTIVE`, and `POWER_ACTIVE`. The power in each FFT bin is smoothed using an exponential moving average (EMA). For bin k in frame m, the averaged power is updated from the current power $P_m[k]$ as

$$A_m[k] = A_{m-1}[k] + 2^{-a_q} \left(P_m[k] - A_{m-1}[k] \right). \tag{2}$$

Where each state q selects an averaging shift a_q and a threshold scale $r^{(q)}$. Setting $a_q = 0$ uses the current frame power, while larger averaging values suppress noise fluctuations to improve weak signal detection. Excessive averaging can delay detected boundaries and merge closely spaced bursts. A bin is declared active when $A_m[k] > \Lambda^{(q)}$, where $\Lambda^{(q)} = r^{(q)} P_N$ scales the calibrated noise reference.

In `IDLE`, parameters favor rapid detection of new activity. A tracked signal triggers `SIGNAL_ACTIVE`. Independently, total frame power is compared with

$$\Lambda_{\mathrm{sum}} = r_{\mathrm{sum}} N P_N, \tag{3}$$

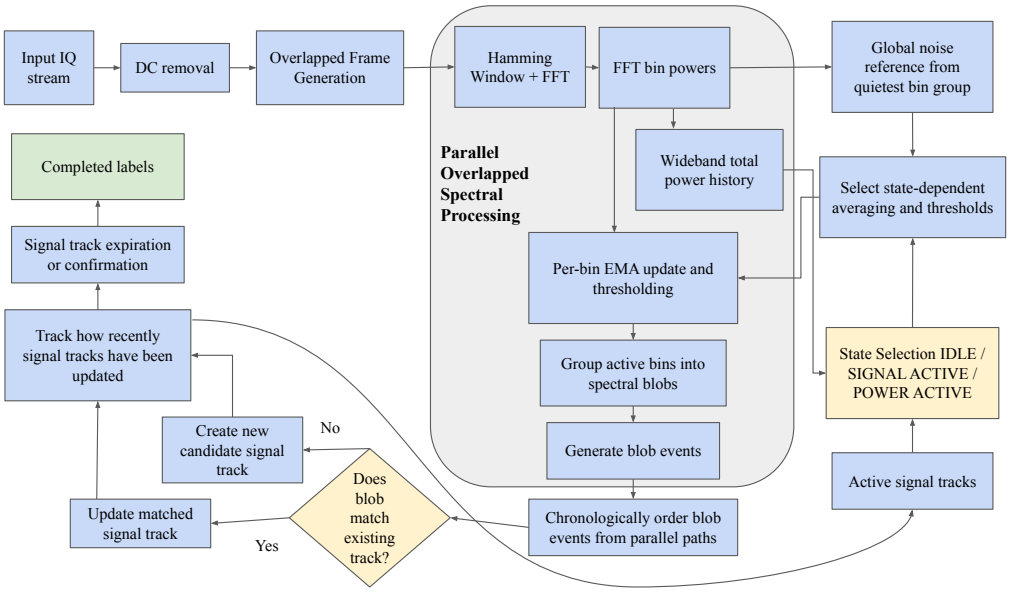


Fig. 1: HI-WHIRLS signal processing flow from IQ input samples to completed time-frequency labels.

where NP_N is the expected frame noise power and r_{sum} is its threshold scale factor. A history of W_{sum} measurements suppresses transitions caused by isolated power fluctuations. Sustained power above Λ_{sum} triggers `POWER_ACTIVE`, selecting alternative averaging and bin-detection thresholds. When this condition clears, the detector returns to `SIGNAL_ACTIVE` if tracked signals remain, or `IDLE` otherwise.

For example, individual transmissions may trigger `SIGNAL_ACTIVE` without substantially increasing total band power. Stronger transmissions can instead trigger `POWER_ACTIVE`, allowing the detector to adjust its settings while continuing to track individual signals. This approach could be extended to multiple power-dependent states, each covering a different total-power range with its own averaging and threshold parameters.

E. Frequency-Domain Grouping

The binary frequency bin decisions are grouped into spectral *blobs*. A spectral blob is a contiguous region of active FFT bins that represents the occupied frequency range of a potential signal within one frame. The algorithm scans the bins from low to high frequency and begins a blob when it encounters an active bin. The blob is extended as additional active bins are observed. A configurable number of inactive bins $G_{\max}$ are allowed without ending the blob. This tolerance prevents spectral nulls, fading, or notches from dividing a single transmission. A candidate blob is retained only when it contains a minimum number of active bins $W_{\min}$. This width requirement suppresses isolated detections that are more likely to have been noise. For each accepted blob b, the algorithm records its lower and upper frequency bin indices.

F. Temporal Signal Tracking

The blobs detected in the current frame are associated with signal tracks retained from previous frames. A blob b matches an existing track t when their frequency ranges overlap by at least one FFT bin. When a blob matches an existing track, the stored frequency boundaries are expanded when necessary and the current frame becomes the track's most recent observation.

An unmatched blob creates a new candidate track. Each successful match increments the track's detection count, and the track is confirmed once the count reaches N_{persist}. This persistence requirement suppresses isolated noise detections. Both candidate and confirmed tracks may tolerate missed detections for up to M_{exp} frames, allowing brief fades before splitting a transmission. Together, $W_{\min}$, $G_{\max}$, N_{persist}, and M_{exp} improve robustness at low SNR by suppressing isolated noise detections and bridging brief gaps in frequency and time. A track expires when

$$m - m_{\mathrm{last}}^{(t)} \geq M_{\mathrm{exp}},$$

where where m is the current frame index and $m_{\mathrm{last}}^{(t)}$ is the frame index of its most recent matching blob. Confirmed expired tracks are defined as completed labeled signals. Each completed signal records its first and last detection frames, m_s and m_e, and its minimum and maximum frequency bins, $k_{\min}$ and $k_{\max}$.

IV. FPGA Implementation

A. System Architecture

HI-WHIRLS is implemented as a continuous streaming pipeline on a Xilinx RFSoC ZCU216 evaluation board, as shown in Fig. 2. The built-in analog to digital converter (ADC) and digital down converter (DDC) supplies complex IQ samples to the programmable logic through AXI4-Stream. A frame router distributes overlapping frames among up to four parallel FFT paths, depending on the configured

TABLE I: Configurable HI-WHIRLS Parameters.

Parameter	Symbol
FFT length	N
Frame overlap	ρ
Calibration frames	M_{cal}
Noise-reference group	G_N
Sum-history length	W_{sum}
Sum-threshold scale	r_{sum}
Minimum active bins	$W_{\min}$
Maximum blob gap	$G_{\max}$
Track persistence	N_{persist}
Track expiration	M_{exp}
EMA shifts	a_q
Detection scales	$r^{(q)}$

overlap. Each path includes an independent streaming blob detector. Detector outputs are buffered in separate FIFOs and merged into a chronologically ordered event stream for the temporal tracker, which maintains active signal records and generates completed labels. The processing system (PS) accesses configuration and status registers through AXI4-Lite. It initializes the Data Converters, programs the FFT cores and HI-WHIRLS parameters, starts the streaming system, and reads status information. The system is also built to be integrated with an external RF front end to receive signals in the 2.425–2.475 GHz band for over-the-air operation.

B. Parallel FFT Processing

The frame router supports 0%, 50%, and 75% overlap using hop sizes of N, $N/2$, and $N/4$, and one, two, or four parallel FFT paths, respectively. Frames are assigned to these paths in round-robin order. For the 75% overlap configuration, a new frame begins every $N/4$ accepted samples. Each engine produces an AXI4-Stream containing the complex FFT outputs, bin indices, and frame boundaries. The streams remain separate through blob detection and are merged after being reduced to events. This avoids reordering the full FFT bin streams at the cost of replicating detector hardware. Higher overlap factors can be supported by adding additional parallel FFT and detector paths, with $\lceil N/H \rceil$ parallel paths required.

C. Detection and Tracking Hardware

1) Calibration and State Control: Each FFT path computes the squared magnitude of its complex output to produce per-bin power values, while the corresponding bin and frame indices are propagated through the pipeline. During calibration, bin powers are accumulated across M_{cal} frames in dual-port block RAM using a synchronous read-modify-write pipeline. After calibration, the state controller uses the total frame power history to select `IDLE`, `SIGNAL_ACTIVE`, or `POWER_ACTIVE`. The corresponding EMA and threshold parameters are then distributed to all detector paths. Detection begins once calibration is complete.

2) Blob Detectors: One streaming blob detector is instantiated for each FFT path and accepts one bin power value per clock cycle. A pipelined read-modify-write datapath reads the previous EMA value from dual-port memory, computes the updated average, compares it with the threshold, and writes the new value back to memory. Active bins are grouped into spectral blobs and written to an output FIFO.

3) Signal Event Ordering: Because each FFT engine processes every fourth frame in the 75% overlap configuration, the local detector frame indices are converted to a common chronological index:

$$m_{\mathrm{global}} = 4m_{\mathrm{local}} + d, \qquad d \in \{0, 1, 2, 3\}, \tag{4}$$

where d is the detector path offset. A minimum-selection tree compares the frame indices at the blob FIFO outputs and forwards the oldest event to the temporal tracker. A global frame reference tracks the latest frame completed by all detector paths so that track expiration is not evaluated before delayed events from earlier frames have been processed.

4) Temporal Signal Tracker: The temporal tracker uses an indexed memory to store each track's confirmation status, frame indices, frequency boundaries, and detection count. A controller searches and updates this table for each incoming blob event, allocating entries for new tracks. The controller also scans the table for expiration using the global frame reference and writes completed, confirmed tracks to the output label FIFO.

D. Configuration, Output, and Hardware Performance

1) AXI4-Lite Interface: The RFSoC PS configures HI-WHIRLS through an AXI4-Lite register interface. The register bank provides control of all the algorithm parameters. AXI4-Lite is also used to read the full signal labels from the label FIFO.

2) Hardware Parameters: The implemented architecture supports configurable FFT lengths from 128 to 8192 points, 0%, 50%, and 75% frame overlap, and up to 16 simultaneous signal tracks.

3) Latency: The end-to-end latency of HI-WHIRLS is measured in Vivado simulation by recording the time between the first sample of the final detected FFT frame and the completed output label. If j is the final detected frame index and H is the hop size,

$$T_{\mathrm{E2E}} = t_{\mathrm{label,out}} - t_{\mathrm{accept}}[jH]. \tag{5}$$

After the final matching detection in frame j, the tracker must advance by M_{exp} frame intervals without another matching blob before the track can expire. Each additional expiration frame therefore adds one hop interval,

$$\frac{H}{f_s} = \frac{128}{50 \times 10^6} = 2.56\ \mu\mathrm{s}.$$

The expiration frame begins $M_{\mathrm{exp}}H$ samples after the beginning of frame j. Since a complete N-sample FFT frame must also be received, the total observation interval is

$$T_{\mathrm{obs}} = \frac{M_{\mathrm{exp}}H + N - 1}{f_s}. \tag{6}$$

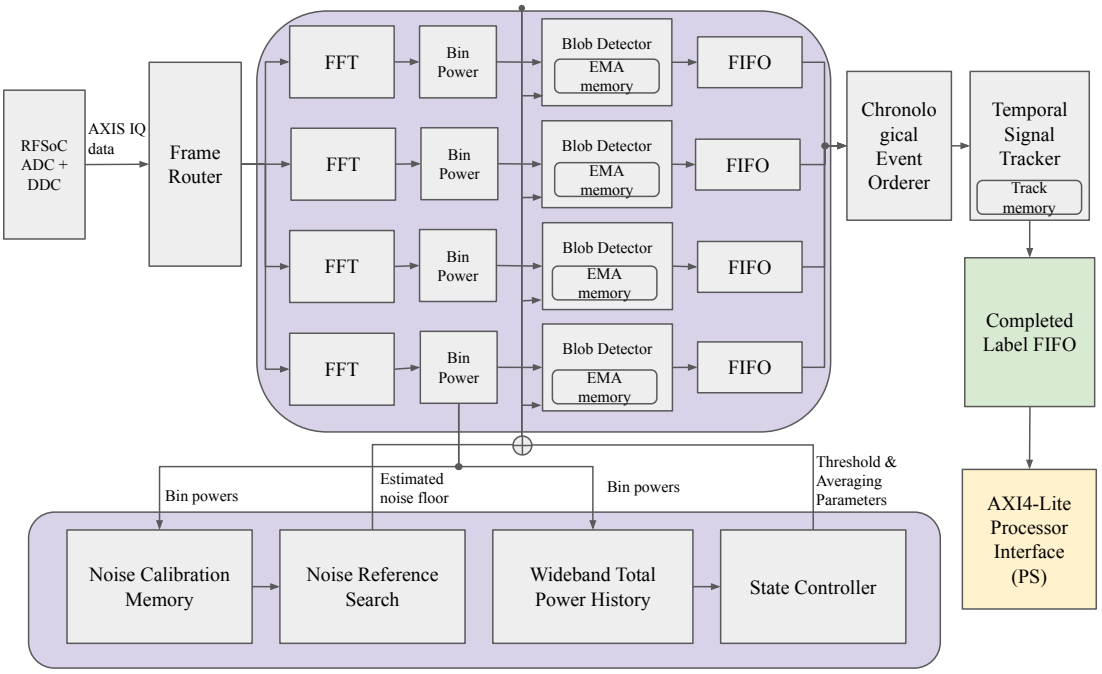


Fig. 2: Complete HI-WHIRLS RFSoC FPGA architecture.

TABLE II: FPGA Resource Utilization and Performance

Metric	Symbol	Result
LUT utilization	—	98,116 (23.1%)
Flip-flop utilization	—	53,740 (6.3%)
BRAM utilization	—	145 (13.4%)
DSP utilization	—	90 (2.1%)
Input sample rate	f_s	50 MSPS
FFT length	N	512 samples
Frame hop size	H	128 samples
Expiration parameter	M_{exp}	5 frames
Measured end-to-end latency	T_{E2E}	38.58 μs

Thus, T_{obs} represents the amount of input data that must arrive after the beginning of the final detected frame before sufficient information is available to declare the track complete. For $N = 512$, $H = 128$, $f_s = 50$ MSPS, and $M_{\text{exp}} = 5$,

$$T_{\text{obs}} = \frac{5(128) + 511}{50 \times 10^6} = 23.02 \ \mu\text{s}.$$

The measured end-to-end latency is $T_{\text{E2E}} = 38.58$ μs, leaving $T_{\text{proc}} = 15.56$ μs for the remaining FPGA processing. The primary contributors to this extra processing latency are the FFT and the pipelined detection and tracking logic with memory accesses.

4) Resource and Performance Results: Post-implementation utilization, and latency are summarized in Table II for an example hardware configuration.

5) Comparison With Prior Real-Time Implementations: Table III compares HI-WHIRLS with prior real-time spectrum sensing and signal localization implementations, focusing on processing latency after the input data is available. Because the systems use different processing architectures and latency definitions, the comparison is not strictly one-to-one.

TABLE III: Comparison With Prior Real-Time Implementations

System	Platform	Input path	Output	Latency
RISE [13]	CPU	SDR to host time-frequency plot	Bounding boxes	16.80 ms
WRIST [9]	CPU–GPU	CPU FFT and compression to GPU	Classes and boundaries	22.96 ms
Wang et al. [17]	FPGA	Buffered samples to FFT and detection	Frequencies and bandwidths	0.25 ms
HI-WHIRLS	RFSoC	ADC directly to programmable logic	Time-frequency labels	38.58 μs

RISE and WRIST incur higher latency because both operate on complete two-dimensional spectral representations before producing a decision. RISE applies several image-processing stages on a multi-threaded CPU, while WRIST performs CPU-based preprocessing followed by neural-network inference on a GPU. These approaches therefore require substantially more computation and buffering per decision than HI-WHIRLS. Wang et al. also use FPGA-based processing, but their architecture employs a much larger 8192-point FFT. HI-WHIRLS instead processes short, overlapping FFT frames continuously as samples arrive in programmable logic. This allows acquisition, detection, and tracking to proceed concurrently.

V. Evaluation Setup

HI-WHIRLS is evaluated using simulations based on the synthetic dataset from the original WHIRLS paper and real-time measurements using the RFSoC implementation.

A. Synthetic Dataset

The synthetic dataset uses the same configuration as the original WHIRLS evaluation [1]. It consists of randomly generated QPSK signals with bandwidths ranging from 0.5 to 30 MHz and durations ranging from 0.2 to 3 ms. Each signal is generated using a raised-cosine filter with roll-off factor $\beta = 0.01$ and a span of 10 symbols. The receiver observes 50 MHz of bandwidth. Each 70 ms realization contains 30 transmissions with known time and frequency boundaries. The signal power is varied to evaluate performance across different SNR conditions, with SNR defined over the full monitored bandwidth during the active signal interval. Each experiment is repeated five times, and the results are averaged.

B. Simulation Description

The simulation model replicates the major processing stages of the FPGA implementation in Python using the same fixed-point interfaces and quantization rules. Simulations use $N = 128$, 75% frame overlap, $M_{\text{cal}} = 50$ calibration frames, and a noise-reference group size of $G_N = 10$ bins. Blob formation uses $W_{\min} = 5$ active bins and a maximum gap of $G_{\max} = 5$ bins. Track confirmation and expiration are increased to $N_{\text{persist}} = 100$ detections and $M_{\text{exp}} = 100$ frames, to suppress noise detections and tolerate brief detection gaps at low SNR. These settings favor stable tracks over rapid confirmation and expiration. The simulation experiments do not stress the adaptive state transitions.

To evaluate the effect of temporal averaging, three EMA shift values are considered, $a \in \{0, 3, 5\}$. The corresponding threshold factor q_a is selected so that each averaging configuration targets the same false-alarm probability of 10^{-3} across the 128 FFT bins. For an EMA shift a, the update weight applied to the current frame is $\alpha = 2^{-a}$. Assuming independent noise samples across frames, the variance reduction produced by the EMA can be expressed as an equivalent number of independently averaged power samples,

$$K_{\text{eff}} = \frac{2 - \alpha}{\alpha} = 2^{a+1} - 1. \tag{7}$$

As K_{eff} increases, the noise-power distribution becomes more concentrated around its mean, and a smaller threshold factor is required to achieve the same false-alarm probability. For noise-only data, the normalized average FFT-bin power is modeled by a Gamma distribution with shape K_{eff} and mean one. The threshold factor q_a is selected from its upper-tail quantile such that

$$\Pr\left(\frac{A_m[k]}{P_N} > q_a\right) = p_{\text{bin}}. \tag{8}$$

For $a = 0$, 3, and 5, the resulting threshold factors are $q_a = 11.7593$, 2.5263, and 1.6401, respectively.

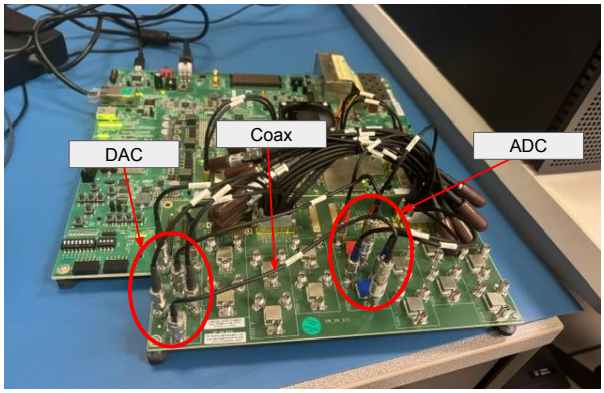


Fig. 3: RFSoC loopback measurement setup used to evaluate HI-WHIRLS.

Two threshold policies are also evaluated. The fixed policy uses

$$R_{\text{fixed}}(a) = q_a, \tag{9}$$

while the SNR-adjusted policy uses

$$R_{\text{adjusted}}(a, \text{SNR}) = q_a \left(1 + \frac{10^{\text{SNR}/10}}{2}\right). \tag{10}$$

The fixed policy isolates the effect of temporal averaging, while the SNR-adjusted policy assumes prior knowledge of the signal SNR and increases the detection threshold to improve localization at higher SNRs.

HI-WHIRLS, WHIRLS [1], RISE [13], and Searchlight [12] are all modeled and evaluated in software using the same synthetic input signals. All four methods use a FFT length of $N = 128$, corresponding to a frequency resolution of 390.625 kHz. HI-WHIRLS and WHIRLS use 75% frame overlap, corresponding to a time resolution of 0.64 μs, while Searchlight and RISE have time resolutions of 2.56 μs. For each detected signal, a detection is considered correct when the detected time-frequency region captures at least 30% of the corresponding ground-truth region. For each algorithm, the SNR is swept from -20 to 10 dB in 1 dB increments, with five trials evaluated at each SNR. Performance is reported using probability of detection p_D, probability of false detections p_{FD}, time RMSE, and frequency RMSE.

C. RFSoC Measurement Setup

The hardware implementation is evaluated using the Xilinx RFSoC ZCU216 in the loopback configuration shown in Fig. 3. Pre-generated QPSK and 16-QAM waveforms, pulse-shaped using a root-raised-cosine filter with a roll-off factor of 0.2 and a span of six symbols, are stored in the FPGA and streamed to the RFSoC DAC to generate the analog RF signal. The DAC output is connected through a coaxial cable to the RFSoC ADC input, forming an analog loopback. The ADC samples the received waveform at 2.0 GSPS, after which digital downconversion and decimation are performed before presenting a 50 MSPS IQ stream to HI-WHIRLS.

VI. Results and Discussion

A. Simulation Results

1) Overall Detection and Estimation Performance: Fig. 4 compares HI-WHIRLS with WHIRLS, RISE, and Searchlight across the evaluated SNR range. HI-WHIRLS uses the fixed and SNR-adjusted threshold policies, both with an EMA shift of $a = 5$, while WHIRLS uses the tight thresholding option described in [1].

As shown in Fig. 4(a), the SNR-adjusted HI-WHIRLS configuration provides strong low-SNR detection performance, similar to Searchlight. HI-WHIRLS approaches perfect detection probabilities consistently around -12 dB and higher SNRs, while WHIRLS and RISE approach perfect detection probability at approximately -5 dB SNR, a 7 dB improvement. The fixed-threshold HI-WHIRLS configuration also detects transmissions at low SNR, but its detection probability becomes less stable as the signal power increases. Temporal averaging enables the low threshold needed for weak signal detection. However, at higher SNR, spectral leakage and sidelobe energy surrounding strong transmissions can exceed the same fixed threshold, causing overestimated bandwidths or spurious tracks. Fluctuations in these marginal detections can also destabilize signal boundaries. The SNR-adjusted policy raises the threshold to reduce these effects. The corresponding false-detection probability is shown in Fig. 4(b) and generally follows the inverse trend of p_D. False detections are highest near each method's low-SNR detection limit and decrease as detection becomes more reliable at higher SNR.

The time localization results in Fig. 4(c) show that the largest errors generally occur near each method's detection limit. RISE, Searchlight, and SNR-adjusted HI-WHIRLS achieve very small time RMSE once detections become reliable, while WHIRLS performs slightly worse. Fixed-threshold HI-WHIRLS exhibits several RMSE peaks that coincide with its less stable detection performance. The frequency localization results in Fig. 4(d) show a clearer separation between the methods. SNR-adjusted HI-WHIRLS achieves the lowest frequency RMSE, while WHIRLS and RISE also reach error below one megahertz. Searchlight, however, maintains several megahertz of frequency error even in its best operating region.

Overall, temporal averaging combined with the SNR-adjusted threshold extends the usable low SNR range of HI-WHIRLS compared to the original WHIRLS implementation, enabling detection results close to Searchlight. Once reliable detection is established, HI-WHIRLS also maintains low time and frequency localization error.

2) Effect of Temporal Averaging: Fig. 5 compares EMA shifts $a \in \{0, 3, 5\}$ for $N = 128$, using the SNR-adjusted threshold policy. Without averaging ($a = 0$), detection improves gradually above 0 dB and approaches unity near 7 dB. In contrast, $a = 3$ and $a = 5$ achieve $p_D = 1$ at approximately -8 dB and -12 dB, respectively. Stronger averaging thus substantially improves low-SNR sensitivity. Although not shown, false detections decrease as detection probability approaches unity, similar to the previous plots. Stronger averaging slightly increases time RMSE by smoothing signal boundaries and can also cause very short transmissions to be missed, although neither are prevalent in these simulated experiments.

B. RFSoC Hardware Results

1) Detection Across SNR: We first examine the performance of the hardware implementation under different received SNR conditions. Fig. 6 evaluates the hardware labeling performance using an FFT length of $N = 512$, 75% frame overlap, corresponding to a frequency resolution of 97.7 kHz and a time resolution of 2.56 μs. HI-WHIRLS uses no temporal averaging for the 10 dB and 0 dB cases, but uses an averaging shift of $a = 3$ for the -10 dB case. HI-WHIRLS successfully detects all transmitted signals across the three SNR conditions, but the estimated time-frequency boundaries become less accurate as the SNR decreases.

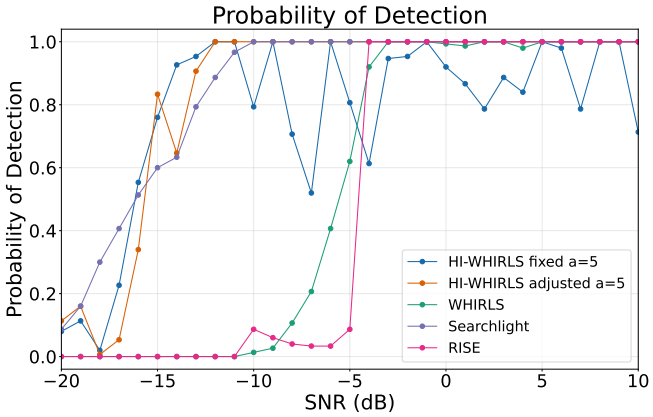
2) Time and Frequency Isolation: We next examine the ability of the RFSoC implementation to isolate closely spaced 10 dB signals in the time and frequency domains. For the time-domain isolation experiment in Fig. 7(a), an FFT length of $N = 128$ with 75% overlap is used, giving a time resolution of 0.64 μs. Transmissions are placed progressively closer together in time. HI-WHIRLS is able to distinguish transmissions separated by substantially less than 0.1 ms. For the frequency-domain isolation experiment in Fig. 7(b), the FFT length is increased to $N = 1024$, corresponding to a frequency resolution of 48.8 kHz. Simultaneous transmissions are placed progressively closer together in frequency, and HI-WHIRLS successfully labeled signals with frequency separations at around 1 MHz.

3) Response to Wideband Interference: Finally, the adaptive state behavior of HI-WHIRLS is evaluated in the presence of wideband interference. The test waveform first contains three localized transmissions at approximately 5 dB, 7 dB, and 10 dB SNR. The same three transmissions are then repeated while a 13 dB wideband interference signal occupies a large fraction of the monitored spectrum.

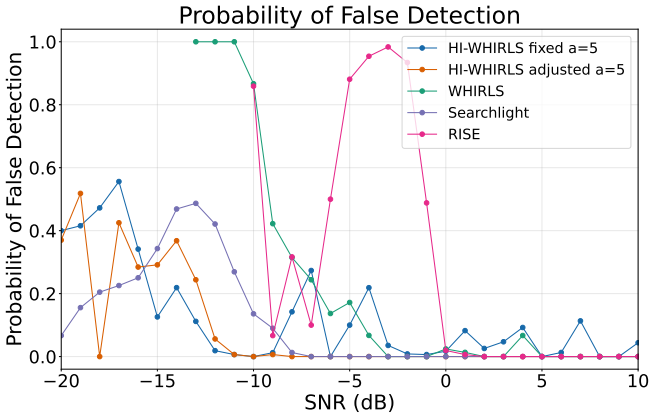
As shown in Fig. 7(c), initial detection of signals initially places the detector in `SIGNAL_ACTIVE`. When wideband interference is introduced, the total received power exceeds the threshold of 13 dB above the calibrated band noise power, triggering `POWER_ACTIVE`. The detector then applies the differently configured averaging and threshold parameters to process the repeated signals of interest. Despite the interference, the transmissions are still detected, although with worsened localization accuracy. This demonstrates HI-WHIRLS' ability to adapt its configuration in real time due to a changing power environment.

VII. Conclusion

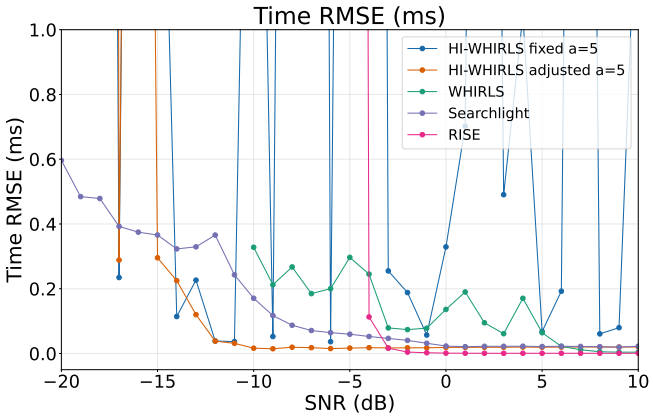
This work presents HI-WHIRLS, a real-time FPGA architecture for wideband signal detection and time-frequency labeling. The architecture implements a continuous streaming pipeline using parallel overlapped FFTs, frequency-domain



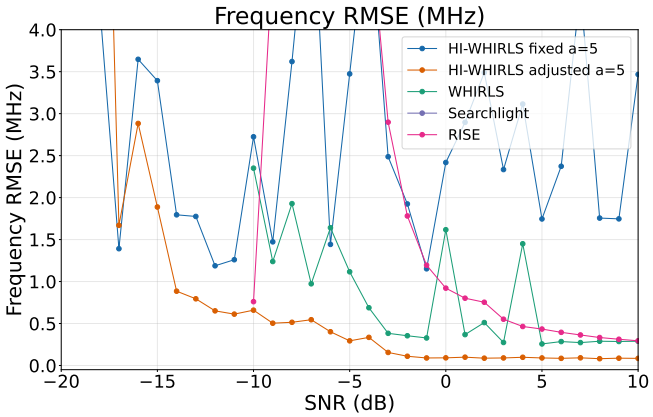
(a) p_D versus SNR



(b) p_{FD} versus SNR



(c) Time RMSE versus SNR



(d) Frequency RMSE versus SNR

Fig. 4: Comparison of simulated HI-WHIRLS, WHIRLS, RISE, and Searchlight on synthetic data

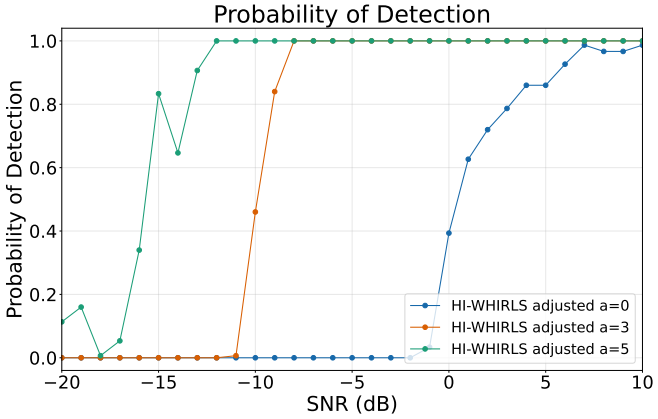


Fig. 5: Effect of temporal averaging on detection probability.

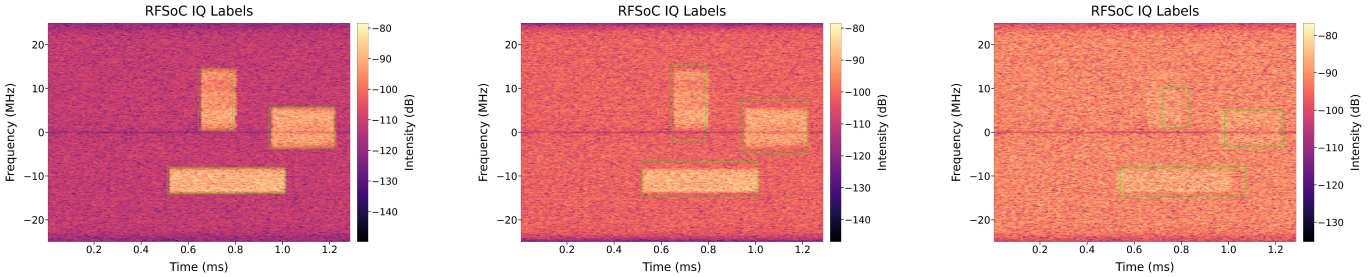
detection, and temporal tracking. Implemented on a Xilinx RFSoC ZCU216, the design supports highly configurable detection parameters, including the time and frequency resolution.

Emulation tests show that HI-WHIRLS maintains accurate time-frequency labeling while improving low-SNR detection relative to the original WHIRLS implementation, with reliable detection down to -12 dB SNR. Hardware loopback evaluations further verify the implemented architecture across RF signals with different SNRs and spacing. For a 512-point FFT, 75% overlap implementation, HI-WHIRLS outputs labels with a measured latency of 38.58 μs, which greatly improves on similar real-time implementations. These results demonstrate that wideband time-frequency labeling can be implemented as a low-latency, continuous-streaming FPGA pipeline suitable for real-time spectrum sensing.

Future work will focus on reducing resource and memory requirements while supporting larger FFT sizes, greater overlap, and higher input sample rates. The current loopback evaluation can also be extended to over-the-air RF measurements. Finally, future work will examine integration with downstream signal classification and spectrum awareness systems so that the generated labels can be used directly for signal identification and monitoring.

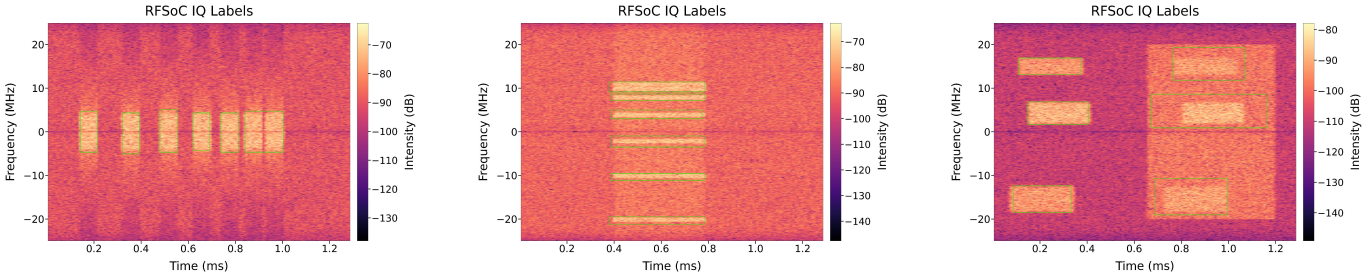
References

[1] T. Zhao, B. W. Domae, C. Steigerwald, L. B. Paradis, T. Chabuk, and D. Cabric, "WHIRLS: Wideband high-resolution labeling of signals



(a) 10 dB SNR (b) 0 dB SNR (c) −10 dB SNR

Fig. 6: RFSoC loopback HI-WHIRLS labels at different SNRs.



(a) Time-domain isolation (b) Frequency-domain isolation (c) Response to wideband interference

Fig. 7: RFSoC time-domain isolation, frequency-domain isolation, and response to wideband interference.

for spectrum sensing applications," in *MILCOM 2025 – 2025 IEEE Military Communications Conference*, pp. 323–329, 2025.

[2] S. A. Saberali and N. C. Beaulieu, "Matched-filter detection of the presence of MPSK signals," in *2014 International Symposium on Information Theory and Its Applications*, pp. 85–89, 2014.

[3] K. Kim, I. A. Akbar, K. K. Bae, J. sun Um, C. M. Spooner, and J. H. Reed, "Cyclostationary approaches to signal detection and classification in cognitive radio," in *2007 2nd IEEE International Symposium on New Frontiers in Dynamic Spectrum Access Networks*, pp. 212–215, 2007.

[4] A. Santra, P. Wang, G. Shaker, B. S. Mysore, G. Dolmans, Y. Chen, N. Shariati, and A. Pandharipande, "Machine learning-powered radio frequency sensing: A review," *IEEE Sensors Journal*, vol. 25, no. 13, pp. 23164–23183, 2025.

[5] T. Yucek and H. Arslan, "A survey of spectrum sensing algorithms for cognitive radio applications," *IEEE Communications Surveys & Tutorials*, vol. 11, no. 1, pp. 116–130, 2009.

[6] D. M. Martínez and Á. G. Andrade, "Adaptive energy detector for spectrum sensing in cognitive radio networks," *Computers & Electrical Engineering*, vol. 52, pp. 226–239, 2016.

[7] N. West, T. Roy, and T. O'Shea, "Wideband signal localization with spectral segmentation," 2021.

[8] K. N. R. S. V. Prasad, K. B. Dsouza, V. K. Bhargava, S. Mallick, and H. Boostanimehr, "A deep learning framework for blind time-frequency localization in wideband systems," in *2020 IEEE 91st Vehicular Technology Conference (VTC2020-Spring)*, pp. 1–6, 2020.

[9] H. N. Nguyen, M. Vomvas, T. D. Vo-Huu, and G. Noubir, "WRIST: Wideband, real-time, spectro-temporal RF identification system using deep learning," *IEEE Transactions on Mobile Computing*, vol. 23, no. 2, pp. 1550–1567, 2024.

[10] T. Zhao, B. W. Domae, C. Steigerwald, L. B. Paradis, T. Chabuk, and D. Cabric, "Drone RF signal detection and fingerprinting: UAVSig dataset and deep learning approach," in *MILCOM 2024 – 2024 IEEE Military Communications Conference*, pp. 431–436, 2024.

[11] T. Cheng, L. Sun, J. Zhang, J. Wang, and Z. Wei, "A start–stop points CenterNet for wideband signals detection and time–frequency localization in spectrum sensing," *Neural Networks*, vol. 170, pp. 325–336, 2024.

[12] R. Bell, K. Watson, T. Hu, I. Poy, F. Harris, and D. Bharadia, "Searchlight: An accurate, sensitive, and fast radio frequency energy detection system," in *MILCOM 2023 – 2023 IEEE Military Communications Conference*, pp. 397–404, 2023.

[13] C.-H. Tung, Z. Qi, and T. Chen, "RISE: Real-time image processing for spectral energy detection and localization," in *2026 IEEE International Symposium on Spectrum Innovation*, pp. 233–242, 2026.

[14] Y. Molina-Tenorio, A. Prieto-Guerrero, and R. Aguilar-Gonzalez, "Real-time implementation of multiband spectrum sensing using SDR technology," *Sensors*, vol. 21, no. 10, p. 3506, 2021.

[15] A. Shrivas and R. Shrestha, "A novel VLSI architecture and FPGA implementation of hardware-efficient MFOCUSS-based wideband spectrum sensor for dynamic spectrum access," in *2026 39th International Conference on VLSI Design and 25th International Conference on Embedded Systems*, pp. 31–36, 2026.

[16] T.-H. Yu, O. Sekkat, S. Rodriguez-Parera, D. Markovic, and D. Cabric, "A wideband spectrum-sensing processor with adaptive detection threshold and sensing time," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 58, no. 11, pp. 2765–2775, 2011.

[17] X. Wang, S. Chaudhari, M. Laghate, and D. Cabric, "Wideband spectrum sensing measurement results using tunable front-end and FPGA implementation," in *2017 51st Asilomar Conference on Signals, Systems, and Computers*, pp. 499–503, 2017.

[18] D. Uvaydov, S. D'Oro, F. Restuccia, and T. Melodia, "DeepSense: Fast wideband spectrum sensing through real-time in-the-loop deep learning," in *IEEE INFOCOM 2021 – IEEE Conference on Computer Communications*, pp. 1–10, 2021.

[19] R. Rajesh, S. J. Darak, A. Jain, S. Chandhok, and A. Sharma, "Hardware–software co-design of statistical and deep-learning frameworks for wideband sensing on Zynq system on chip," *IEEE Transactions on Very Large Scale Integration Systems*, vol. 31, no. 1, pp. 79–89, 2023.

[20] W. Ali, A. Kamaleldin, D. Göhringer, and M. A. A. E. Ghany, "Quantized semantic segmentation for efficient spectrum sensing on FPGAs," in *2025 14th International Conference on Modern Circuits and Systems Technologies*, pp. 1–4, 2025.